LSIS-AFS Reference Implementation — Deliverables, Timeline & Gateway Requirements

Overview

This document defines the required deliverables, timeline, and gateway requirements for the LSIS-AFS reference implementation competition. The project is structured using a **gateway-based approach** where each gateway represents a functional capability that must be demonstrated and validated against the specification.

What is a Gateway? A gateway is a development milestone that delivers a complete, independently testable capability. Each gateway: - Produces tangible outputs (code, test vectors, documentation) - Has clear success criteria for validation - Builds upon previous gateways - Can be evaluated independently

Why This Approach? - **Measurable Progress**: Clear checkpoints throughout the competition - **Quality Focus**: Validate each component before integration - **Partial Credit**: Demonstrate value even if not all gateways complete - **Risk Reduction**: Identify issues early when they're easier to fix

---

Competition Key Dates

Date	Milestone
5 April 2026	Registration & Team Formation Closes
Late June / Early July 2026	Workshop at Goonhilly Earth Station, Cornwall, UK
31 August 2026	Competition Closes — Final Submission Deadline

---

Suggested Timeline

There are no hard intermediate deadlines — all deliverables are due at final submission on 31 August 2026. The gateways below are progressive milestones to help you plan and track progress.

Phase	Period	Gateways	Focus
Foundation	April 2026	0, 1	Architecture, spreading code generation
Encoding	May 2026	2, 3	FEC implementation, message framing
Signal	May–June 2026	4	Baseband I/Q signal generation
Workshop Prep	Late June 2026	—	Demo encode pipeline, interop testing with other teams
Decoding	July 2026	5, 6	Frame sync, decoding, message parsing

Phase	Period	Gateways	Focus
Integration	August 2026	7, 8	End-to-end validation, documentation, final polish

---

Minimum Viable Submission (Pass/Fail Gate)

Your submission must include all of the following or it receives 0 points and is not evaluated:

1. Generate at least 12 PRN spreading codes correctly
2. Encode a complete 12-second navigation frame
3. Generate I/Q signal files
4. Pass validation tests for implemented features
5. Include setup/build instructions

Scoring Overview (100 points)

Category	Points
Correctness	40
Performance	20
Completeness	20
Code Quality	10
Innovation & Extras	10

Gateway 0: Design & Architecture

Target: Month 1 (April 2026) **Goal**: Design system architecture and establish development foundation.

Deliverables

- System architecture document
- Component interaction diagrams
- Technology stack selection
- Development environment setup
- Testing strategy and framework
- Code structure and module design
- Interface definitions
- Data format specifications

Success Criteria

- Clear architecture with defined components
- Development environment ready
- Testing framework established

- Team aligned on approach

Gateway 1: Spreading Code Generation

Target: Month 2 (May 2026) **Goal**: Generate all spreading codes per LSIS Section 2.3.5. **Scoring weight**: 10 points (Correctness)

Deliverables

- Gold code generator producing 2046-chip codes for PRN 1–210
- Legendre sequence generator
- Weil primary code generator producing 10230-chip codes for PRN 1–210
- Weil tertiary code generator producing 1500-chip codes for PRN 1–210
- Secondary code generator (4 chips, 4 variants per Table 10)
- Tiered code assembly with coherent generation
- Validation against Annex3 reference codes
- Test suite covering all 210 PRN codes

Requirements

ID	Requirement
FR-1.1	Generate Gold codes (2046 chips) for PRN 1–210
FR-1.2	Generate Weil primary codes (10230 chips) for PRN 1–210
FR-1.3	Generate secondary codes (4 chips, 4 variants)
FR-1.4	Generate Weil tertiary codes (1500 chips) for PRN 1–210
FR-1.5	Tiered code assembly with coherent generation

Validation Checklist

- ☐ Gold codes for all 210 PRNs match Annex3 hex references exactly
- ☐ Weil primary codes for all 210 PRNs match Annex3 references exactly
- ☐ Weil tertiary codes for all 210 PRNs match Annex3 references exactly
- ☐ Secondary codes match Table 10
- ☐ Tiered codes maintain coherency
- ☐ Code lengths match Table 9 specifications

Success Criteria

- All codes match reference hexadecimal values from Annex3
- Code generation completes in < 1 second per PRN
- All 12 interim test codes (Table 11) working
- 100% test coverage for code generators

Gateway 2: Forward Error Correction (Encoding Infrastructure)

Target: Month 3 (June 2026) **Goal**: Implement all FEC codes per LSIS Section 2.4. **Scoring weight**: Part of 10 points (Frame Encoding — Correctness)

Deliverables

- BCH(51,8) encoder and decoder with generator polynomial 763
- CRC-24 generator and validator with specified polynomial
- LDPC encoder and decoder (rate 1/2) for SB2, SB3, SB4
- Block interleaver and deinterleaver (60×98)
- Test vectors for each encoder

Requirements

ID	Requirement
FR-2.1	BCH(51,8) encoding for Subframe 1
FR-2.2	LDPC(1/2) encoding for Subframes 2, 3, 4
FR-2.3	CRC-24 generation for error detection
FR-2.4	Block interleaving (60×98)

Validation Checklist

- ☐ BCH encoder produces valid 52-symbol codewords
- ☐ LDPC encoder produces valid codewords and handles puncturing correctly
- ☐ CRC-24 computation matches specification
- ☐ Interleaver pattern validated
- ☐ Decoders recover original data via round-trip
- ☐ BER < 10^{-5} at SNR > 0 dB

Success Criteria

- Encoding completes in < 100ms per frame
- Decoding completes in < 1 second per frame
- BER < 10^{-5} at SNR > 0 dB

Gateway 3: Navigation Message Framing (Message Builder)

Target: Month 3 (June 2026) **Goal**: Build complete 12-second navigation frames per LSIS Section 2.4. **Scoring weight**: Part of 10 points (Frame Encoding — Correctness)

Deliverables

- Synchronization pattern generator (68 symbols)
- Subframe 1 builder (FID + TOI, BCH encoded)
- Subframe 2 builder (Clock & Ephemeris, LDPC encoded)
- Subframe 3 builder (Variable data, LDPC encoded)
- Subframe 4 builder (Network access, LDPC encoded)
- Frame assembler with sync pattern
- Complete 12-second frame generator
- Frame export in multiple formats

Requirements

ID	Requirement
FR-2.5	Frame assembly with sync pattern
—	Frame structure matches Figure 9
—	Bit allocations match Tables 14, 18, 19, 20
—	Symbol counts: SP(68) + SB1(52) + interleaved(SB2+SB3+SB4)(5880) = 6000
—	Frame timing is exactly 12 seconds

Validation Checklist

- ☐ Frame structure matches Figure 9
- ☐ Symbol counts: 68 + 52 + 5880 = 6000
- ☐ Bit allocations match specification tables
- ☐ Frame duration is 12 seconds

Success Criteria

- Generate valid frames with all subframes
- Correct bit allocations per specification
- Frame timing matches 12-second requirement

Gateway 4: Baseband Signal Generation

Target: Month 4 (July 2026) **Goal**: Generate I/Q baseband samples per LSIS Section 2.3. **Scoring weight**: Part of 10 points (Signal Gen/Decode — Correctness)

Deliverables

- Tiered code assembly (primary + secondary + tertiary)
- BPSK modulator for AFS-I (1.023 Mchip/s)
- BPSK modulator for AFS-Q (5.115 Mchip/s)
- I/Q sample generator with configurable sample rate
- Code synchronization implementation
- Signal file export (binary, CSV)

Requirements

ID	Requirement
FR-3.1	AFS-I baseband at 1.023 Mchip/s
FR-3.2	AFS-Q baseband at 5.115 Mchip/s
FR-3.3	BPSK modulation (logic 1 –> -1.0, logic 0 –> +1.0)
FR-3.4	I/Q sample generation at configurable rates
FR-3.5	Signal file export in multiple formats

Validation Checklist

- ☐ I/Q samples correctly formatted
- ☐ Chip rates: 1.023 Mchip/s (AFS-I) and 5.115 Mchip/s (AFS-Q)
- ☐ Symbol rate: 500 symbols/s for AFS-I (per Table 7)
- ☐ Code synchronization meets specification
- ☐ Signal duration matches frame duration (12 seconds)

Success Criteria

- Generate 12-second I/Q signal files
- Correct chip rates and symbol rates
- I/Q samples ready for RF simulation / testing

Gateway 5: Frame Synchronization & Decoding

Target: Month 5 (August 2026) **Goal**: Detect and decode frames from received signals. **Scoring weight**: Part of 10 points (Signal Gen/Decode — Correctness)

Deliverables

- Frame synchronization via sync pattern detection
- Symbol extraction from I/Q samples
- BCH(51,8) soft-decision decoder
- LDPC belief propagation decoder
- CRC validator
- Block deinterleaver

Requirements

ID	Requirement
FR-4.1	Frame synchronization via sync pattern
FR-4.2	BCH(51,8) soft-decision decoding
FR-4.3	LDPC belief propagation decoding
FR-4.4	CRC validation
FR-4.5	Block deinterleaving

Validation Checklist

- ☐ Frame sync detection > 99% at SNR > 0 dB
- ☐ Decoders recover original data correctly
- ☐ CRC validation catches errors
- ☐ LDPC decoder converges in < 50 iterations

Success Criteria

- Decode frames in < 1 second
- BER < 10^{-5} at SNR > 0 dB
- Frame sync detection > 99% reliability at SNR > 0 dB

Gateway 6: Message Parsing

Target: Month 5 (August 2026) **Goal**: Extract navigation data from decoded frames.

Deliverables

- Subframe 1 parser (FID + TOI extraction)
- Subframe 2 parser (Clock & Ephemeris)
- Subframe 3 parser (Variable data routing)
- Subframe 4 parser (Network access)
- Time of Transmission calculator
- Message field extractors for all fields

Requirements

ID	Requirement
FR-5.1	Extract FID and TOI from Subframe 1
FR-5.2	Parse Clock & Ephemeris from Subframe 2
FR-5.3	Route variable data from Subframe 3
FR-5.4	Parse network access from Subframe 4
FR-5.5	Calculate Time of Transmission

Validation Checklist

- ☐ All subframes parse correctly
- ☐ WN, ITOW, TOI fields extracted
- ☐ Time of transmission calculated accurately
- ☐ Time reconstruction accurate to code phase
- ☐ All message types in SB3/SB4 handled

Success Criteria

- Parse all subframe types correctly
- Calculate time of transmission accurately
- Handle all message types

Gateway 7: Integration & Validation

Target: Month 6 (August 2026) **Goal**: Demonstrate end-to-end functionality, interoperability, and compliance.

Deliverables

- Round-trip tests (encode –> signal –> decode –> verify)
- Interoperability test results with other implementations
- Performance benchmarks (throughput, latency)
- Compliance validation report
- Test vector suite
- BER vs SNR performance curves

Requirements

ID	Requirement
FR-6.1	Generate test vectors for all components
FR-6.2	Validate against reference codes
FR-6.3	Measure BER vs SNR performance
FR-6.4	Verify LSIS compliance

Validation Checklist

- ☐ Round-trip recovers original data with 100% accuracy
- ☐ Performance meets all NFR-1 targets
- ☐ All 12 interim test codes (Table 11) working
- ☐ All specification "shall" requirements verified
- ☐ Interoperability with at least one other implementation demonstrated

Success Criteria

- End-to-end pipeline works correctly
- Round-trip testing passes with 100% accuracy
- Process 12-second frames in < 1 second
- All compliance checks pass
- Successful interoperability testing

Gateway 8: Documentation & Examples

Target: Month 6 (August 2026) **Goal**: Provide complete documentation for using and understanding the implementation.

Deliverables

- README with project overview
- Setup and build instructions (reproducible)
- API documentation
- Usage examples for all components
- Design documentation with architecture decisions
- Test documentation and how to run tests
- Performance analysis report

Validation Checklist

- ☐ Setup instructions complete — anyone can build and run from scratch
- ☐ Usage examples work and demonstrate key functionality
- ☐ All public APIs documented
- ☐ Tests documented with instructions to reproduce
- ☐ All dependencies documented or included

Success Criteria

- Anyone can build and run the implementation following instructions
- All features are documented
- Examples demonstrate key functionality
- Reproducible results

Final Submission Package (31 August 2026)

Required

1. Complete source code
2. Build/setup instructions
3. Test suite with results
4. Documentation (setup, usage, API)
5. Examples demonstrating functionality
6. Validation report showing compliance

Optional but Recommended

- Performance analysis
- Design documentation
- Additional test vectors
- Visualization tools
- BER performance curves

Validation Data to Include

- Reference codes from Annex3 (or instructions to obtain)
- LDPC matrices from Annex1 (or instructions to obtain)
- Test vectors you generated
- Code generation validation results
- Frame encoding/decoding test results
- Compliance checklist

Non-Functional Requirements Summary

ID	Requirement	Target
NFR-1.1	Code generation time	< 1 second per PRN
NFR-1.2	Frame encoding time	< 100ms per frame
NFR-1.3	Frame decoding time	< 1 second per frame
NFR-1.4	Real-time capability	Process faster than signal duration
NFR-2.1	Spreading code accuracy	100% match with Annex3
NFR-2.2	BER at SNR > 0 dB	$< 10^{-5}$
NFR-2.3	Frame sync reliability	> 99% at SNR > 0 dB
NFR-2.4	Time reconstruction	Accurate to code phase
NFR-3.1	Unit tests	All major components
NFR-3.2	Integration tests	End-to-end flow
NFR-3.3	Test coverage	> 90%
NFR-3.4	Test vectors	Reproducible

Non-Functional Requirements: Usability

ID	Requirement	Target
NFR-4.1	Clear API documentation	All public APIs documented
NFR-4.2	Working examples	Examples for all features
NFR-4.3	Error messages	Actionable guidance
NFR-4.4	Configurable parameters	Runtime configuration support

Non-Functional Requirements: Maintainability

ID	Requirement	Target
NFR-5.1	Modular architecture	Clean component boundaries
NFR-5.2	Separation of concerns	Independent modules
NFR-5.3	Consistent coding style	Enforced via tooling
NFR-5.4	Version control	Git with meaningful history

Technical Constraints

TC-1: Language & Tools

- Language choice is flexible (e.g., Python, Rust, C++, Go, Java)
- Use appropriate open source libraries for numerical operations and signal processing
- Standard library or equivalent for core algorithms
- Proprietary libraries discouraged for accessibility and reproducibility

TC-2: Data Formats

- Binary I/Q samples (int16, float32)
- CSV for analysis and debugging
- JSON for configuration
- HDF5 for large datasets (optional)

TC-3: Dependencies

- LDPC matrices from Annex1 CSV files
- Reference codes from Annex3 files
- LSIS specification document

TC-4: Compatibility

- Cross-platform (Windows, Linux, macOS)
- No RF hardware dependencies
- Standalone operation (no external services)
- Open source dependencies preferred

Risk Assessment

Risk	Impact	Mitigation
LDPC decoder complexity	High	Use existing open source libraries or simplified algorithm
Test vector availability	Medium	Generate own test vectors from encoder
Performance bottlenecks	Medium	Profile and optimize critical paths
Specification ambiguity	Low	Reference NASA/ESA GNSS implementations
Time constraints	High	Prioritize core functionality, defer optional features

---

Success Metrics

1. **Completeness**: All 8 gateways delivered
2. **Correctness**: Pass all compliance tests
3. **Performance**: Meet all NFR-1 targets
4. **Quality**: Test coverage > 90%
5. **Usability**: Complete documentation and examples

Flexibility & Constraints

You Have Complete Freedom In

- Programming language choice
- Architecture and design
- Algorithms and implementation
- Tools and libraries
- File organization

You Must Comply With

- LSIS specification (all “shall” requirements)
- Output correctness (match reference data)
- Performance targets (timing, BER)
- Deliverable completeness (all gateways)

Your Implementation Will Be Validated By

- Comparing outputs against Annex references
- Testing with known inputs
- Measuring performance
- Checking specification compliance